

Application Based Questions.

1) Online Video Streaming platform.

Given recurrence :

$$T(n) = 2T(n/2) + n$$

Identify the parameters,

$$T(n) = aT(n/b) + f(n)$$

where $a=2$, $b=2$, $f(n)=n$

$$n \log_b a = n \log_2 2 = n$$

Apply Master Theorem,

$$f(n) = \Theta(n \log_b a) = \Theta(n).$$

$$\therefore T(n) = \Theta(n \log n)$$

Time Complexity ($\Theta(n \log n)$)

Asymptotic Interpretation:

$$\text{Big-O} : \Theta(n \log n)$$

$$\text{Big-Theta} : \Theta(n \log n)$$

$$\text{Big-Omega} : \Omega(n \log n)$$

2) Social media Feed processing

Given recurrence :

$$T(n) = T(n-1) + n$$

Expand recurrence,

$$T(n) = T(n-1) + n$$

$$= T(n-2) + (n-1) + n$$

$$= T(n-3) + (n-2) + (n-1) + n$$

continuing the expansion,

$$= T(1) + 2 + 3 + \dots + n$$

The Sum of first n natural numbers is

$$\frac{n(n+1)}{2}$$

$$T(n) = T(1) + \frac{n(n+1)}{2}$$

Ignoring constant term,

$$T(n) = \Theta(n^2)$$

Time Complexity : $\Theta(n^2)$

4) Cloud Storage Processing System

Given, recurrence

$$T(n) = 3T(n/2) + n$$

Identify parameters,

$$T(n) = aT(n/b) + f(n)$$

where, $a = 3$, $b = 2$, $f(n) = n$

calculate: $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$

Apply Master Theorem,

$$f(n) = O(n^{\log_2 3 - \epsilon})$$

Therefore, $T(n) = O(n^{\log_2 3})$

Time Complexity: $O(n^{1.585})$

The Cloud Storage processing alg. has $O(n^{1.585})$ time complexity. It offers better scalability than quadratic algorithms, making it suitable for processing large storage system while maintaining acceptable performance.

3) E-commerce Recommendation System.

The system uses two algorithms:

$\swarrow$
 $O(n \log n)$ $\searrow$ $O(n^2)$

Growth rate Analysis:

$O(n \log n)$: Grows slowly and is suitable for large datasets.

$O(n^2)$: Grows rapidly because every increase in input size greatly increases the no. of operations.

Efficiency:

The $O(n \log n)$ algorithm is more efficient than $O(n^2)$ algorithm because it requires significantly fewer operations.

Conclusion:

For large scale e-commerce recommendation system, $O(n \log n)$ algorithm is better choice. It provides faster execution, better scalability, & improved overall performance, whereas $O(n^2)$ becomes inefficient for large datasets.

5. Search Engine Index Builder

Given recurrence:

$$T(n) = T(n/2) + 1$$

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 1 + 1 = T(n/8) + 1 + 1 + 1$$

After k expansions,

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

The recursion stops when

$$n/2^k = 1$$

$$2^k = n$$

$$k = \log_2 n$$

Substitute into recurrence,

$$T(n) = T(1) + \log_2 n$$

Ignoring constant = $\Theta(\log n)$

⇒ The input size is reduced by half at each recursive call.

⇒ The no. of recursive calls grows logarithmically

⇒ Even for very large datasets, execution time increases slowly, making algorithm highly scalable.

6. Gaming Engine Optimization.

The Gaming engine compares three

algorithms:

- 1) $O(n)$
- 2) $O(n \log n)$
- 3) $O(n^2)$

Comparison			
Input Size	$O(n)$	$O(n \log n)$	$O(n^2)$
100	100	≈ 664	10,000
1000	1000	≈ 9966	1,000,000
10000	10000	$\approx 132,877$	100,000,000

$O(n)$: linear growth; execution time increases with $1/p$

$O(n \log n)$: slightly faster growth than linear but remains efficient for large inputs.

$O(n^2)$: Quadratic growth, execution time increase rapid for high-resolution graphics.

Conclusion:

For high resolution gaming inputs, $O(n)$ alg provides best performance & scalability. The $O(n \log n)$ algorithm is also practical, while $O(n^2)$ algorithm becomes inefficient as input size.

Data Backup Optimization System.

$$T(n) = 2T(n/2) + \sqrt{n}$$

Identify parameters,

$$T(n) = aT(n/b) + f(n)$$

Where : $a = 2$, $b = 2$, $f(n) = \sqrt{n} = n^{1/2}$

$$n^{\log_b a} = n^{\log_2 2} = n$$

Compare $f(n)$ & $n^{\log_b a}$,

$$\sqrt{n} = n^{1/2}$$

$$n^{1/2} = O(n^{1-\epsilon})$$

for $\epsilon = \frac{1}{2}$, Time Complexity = $O(n)$

Performance Analysis,

⇒ The recursive work dominates additional work.

⇒ The running time grows linearly with I/P size.

⇒ The alg scales efficiently for every large backup datasets.